

**CONCEITOS-
-CHAVE**

conhecimento	723
ética.	725
espectro de	
informações	723
futuro.	721
profissionais	722
software revisitado .	722

Nos 31 capítulos deste livro, exploramos um processo para engenharia de software que abrange tanto procedimentos de gestão quanto métodos técnicos, conceitos básicos e princípios, técnicas especializadas, atividades e tarefas orientadas a pessoas, que são próprias à automação, anotações com lápis e papel e ferramentas de software. Argumentamos que medições, disciplina e um foco na agilidade e qualidade resultarão em software que atenda às necessidades do cliente, software que é confiável, software que é manutenível, um software melhor. Contudo, nunca foi dito que a engenharia de software é um remédio para todas as dificuldades.

Enquanto caminhamos para a segunda década deste século, as tecnologias de software e sistemas permanecem um desafio para todos os profissionais de software e todas as empresas que criam sistemas baseados em computadores. Embora tenhamos escrito essas palavras com a visão do século 20, Max Hopper [Hop90] descreve com precisão o estado atual das coisas:

Uma vez que as mudanças na tecnologia de informação estão se tornando tão rápidas e intormentantes e as consequências de ficar para trás são tão irreversíveis, ou as empresas dominam a tecnologia ou morrem... Pense nisso como uma escravidão tecnológica (technology treadmill). As empresas terão de lutar cada vez mais para manter sua posição.

As mudanças na tecnologia de engenharia de software são sem dúvida “rápidas e inclementes”, ao mesmo tempo o progresso é com frequência bastante lento. Até que seja tomada a decisão de adotar um novo processo, método ou ferramenta, seja realizado o treinamento necessário para entender sua aplicação e seja introduzida a tecnologia na cultura de desenvolvimento de software, algo mais novo (e até melhor) surge, e o processo todo começa novamente.

Uma coisa que aprendemos durante muitos anos neste campo é que os profissionais de engenharia de software são “conscientes de sua situação”. O caminho à frente estará entulhado de carcaças de novas e excitantes tecnologias (a última moda) que nunca tiveram sucesso (apesar da quantidade de “novidades”). O futuro será formado por tecnologias mais modestas que de certa forma modificam a direção e a amplitude do caminho. Abordei algumas dessas tecnologias no Capítulo 31.

Neste capítulo conclusivo vamos adotar uma visão mais ampla e considerar onde estamos e para onde estamos indo a partir de uma perspectiva mais filosófica.

PANORAMA

O que é? Ao chegarmos ao fim de uma jornada relativamente longa pela engenharia de software, é hora de colocarmos os fatos em perspectiva e fazer alguns comentários finais.

Quem realiza? Autores como eu. Quando você chega ao fim de um livro longo e desafiador, é ótimo poder reunir os fatos de maneira clara.

Por que é importante? É sempre bom lembrar onde estivemos e pensar para onde vamos.

Quais são as etapas envolvidas? Farei considerações sobre o que vimos e vou tratar de alguns dos

assuntos importantes e algumas direções para o futuro.

Qual é o artefato? Uma discussão que o ajudará a entender o contexto real de desenvolvimento de software.

Como garantir que o trabalho foi realizado corretamente? Isso é difícil de conseguir em tempo real. Somente daqui a alguns anos é que se pode dizer se conceitos, princípios, métodos e técnicas de engenharia de software discutidos neste livro o ajudaram a tornar-se um engenheiro de software melhor.

32.1 A IMPORTÂNCIA DO SOFTWARE REVISITADA

A importância do software pode ser definida de várias maneiras. No Capítulo 1, o software foi caracterizado como um diferenciador. A função proporcionada pelo software diferencia produtos, sistemas e serviços e traz uma vantagem competitiva no mercado. Mas o software é mais do que um diferenciador. Quando considerados como um todo, os produtos de engenharia de software geram o bem de consumo mais importante que qualquer indivíduo, negócio ou governo pode adquirir – a informação.

No Capítulo 31, abordou-se rapidamente a computação aberta – inteligência ambiente, aplicações ligadas a contexto e computação invasiva/onipresente –, uma direção que mudará fundamentalmente nossa percepção de computadores, as coisas que fazemos com eles (e que eles fazem para nós) e nossa percepção das informações como um guia, um bem e uma necessidade. Destaquei também que o software necessário para suportar a computação aberta apresentará novos desafios para os engenheiros de software. Porém mais importante ainda, a futura invasão do software de computador apresentará desafios ainda mais significativos para a sociedade como um todo. Sempre que uma tecnologia tem um amplo impacto – que pode salvar vidas ou colocá-las em perigo, criar negócios ou destruí-los, informar os líderes dos governos ou confundi-los –, deve ser “manuseada com cuidado”.

32.2 PROFISSIONAIS E A MANEIRA COMO CONSTROEM SISTEMAS

“Choque do futuro é o ambiente de estresse e desorientação que criamos ao redor dos indivíduos, submetendo-os a inúmeras mudanças em um período de tempo muito curto.”

Alvin Toffler

O software necessário para os sistemas de alta tecnologia torna-se cada vez mais complexo a cada ano que passa, e o tamanho dos programas resultantes aumenta proporcionalmente. O rápido crescimento do tamanho dos programas “médios” não nos causaria problemas se não fosse por um simples fato: à medida que aumenta o tamanho do programa, o número de pessoas envolvidas com o trabalho também aumenta.

A experiência mostra que quando o número de profissionais em uma equipe de projeto de software aumenta, a produtividade geral do grupo pode ficar prejudicada. Um meio de contornar o problema está na criação de uma série de equipes de engenharia de software, dividindo assim as pessoas em grupos individuais de trabalho. No entanto, enquanto o número de equipes de engenharia de software cresce, a comunicação entre elas se torna tão difícil e morosa quanto a estabelecida entre os indivíduos. Pior ainda, a comunicação (entre indivíduos ou equipes) tende a ser ineficiente – muito tempo é gasto transferindo-se pouco conteúdo e, com frequência, informações importantes “caem no vazio”.

Se a comunidade de engenharia de software tem de enfrentar efetivamente o dilema das comunicações, o caminho para os engenheiros de software deve incluir mudanças radicais na maneira como os profissionais e as equipes se comunicam uns com os outros. No Capítulo 31, abordei os ambientes colaborativos que podem proporcionar melhorias significativas na maneira como as equipes se comunicam.

Na análise final, a comunicação é a transferência de conhecimento, e a aquisição (e transferência) de conhecimento está mudando profundamente. Conforme os dispositivos de busca se tornam cada vez mais sofisticados e as aplicações para Web 2.0 proporcionam uma melhor sinergia, a maior biblioteca do mundo de artigos de pesquisa e relatórios, tutoriais, comentários e referências torna-se facilmente acessível e utilizável.

Se a história pode servir de referência, podemos afirmar que as pessoas em si não mudarão. No entanto, a maneira pela qual elas se comunicam, o ambiente no qual trabalham, o modo como adquirem conhecimentos, os métodos e ferramentas que utilizam, a disciplina que aplicam e, portanto, a cultura geral para o desenvolvimento futuro mudarão de forma significativa e ainda mais profunda.

32.3 NOVOS MODOS DE REPRESENTAR AS INFORMAÇÕES

“A melhor preparação para fazer um bom trabalho amanhã é fazer um bom trabalho hoje.”

Elbert Hubbard

Na história da computação, houve uma mudança sutil na terminologia usada para descrever o trabalho de desenvolvimento de software para a comunidade dos negócios. Há 40 anos, o termo *processamento de dados* era a frase operativa para descrever o uso dos computadores em um contexto comercial. Hoje, o processamento de dados deu origem a outra expressão – tecnologia da informação –, que implica a mesma coisa, mas apresenta uma mudança sutil no foco. A ênfase não é apenas no processamento de grandes quantidades de dados, e sim na extração de informações importantes desses dados. Obviamente, essa sempre foi a intenção, mas a transformação na tecnologia reflete uma alteração muito mais importante na filosofia de gerenciamento.

Quando as aplicações de software são discutidas hoje, as palavras *dados*, *informações* e *conteúdo* ocorrem repetidamente. Encontramos a palavra *conhecimento* em algumas aplicações de inteligência artificial, mas seu uso é relativamente raro. Praticamente ninguém discute sabedoria no contexto de aplicações de software.

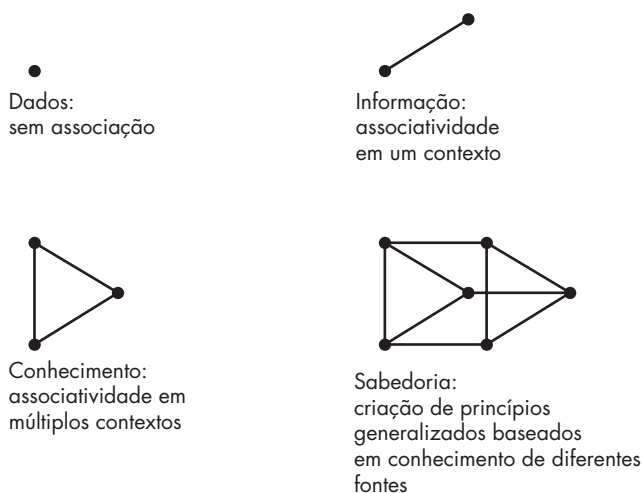
Dados são informações em estado bruto – coleções de fatos que devem ser processadas para ter um significado. As informações surgem associando-se fatos em determinado contexto. Conhecimento associa informações obtidas em um contexto com informações obtidas em outro. Por fim, a sabedoria ocorre quando princípios generalizados são derivados de conhecimentos dispersos. Cada uma dessas quatro visões da “informação” é representada esquematicamente na Figura 32.1.

Até hoje, grande parte do software tem sido criada para processar dados ou informações. Engenheiros de software estão agora igualmente preocupados com sistemas que processam conhecimento.¹ Conhecimento é bidimensional. Informações coletadas sobre uma variedade de tópicos relacionados e não relacionados são conectadas para formar um corpo do fato que chamamos de *conhecimento*. A chave é nossa habilidade em associar informações de uma variedade de origens que podem não ter qualquer conexão óbvia e combiná-las de maneira que nos proporcione um benefício distinto.²

Para ilustrar a progressão dos dados para o conhecimento, considere os dados do senso indicando que o número de nascimentos em 1996 nos Estados Unidos foi de 4,9 milhões. Esse número representa o valor de um dado. Relacionando esse dado com os números de nascimentos nos 40 anos anteriores, podemos gerar uma informação útil – os *baby boomers* dos anos 1950 e início

FIGURA 32.1

Um espectro de “informações”



¹ O rápido crescimento das tecnologias de obtenção de dados e tratamento de dados reflete essa tendência em crescimento.

² A semântica da Web (Web 2.0) permite a criação de “mashups” que podem proporcionar um mecanismo fácil para conseguir isso.

“Sabedoria é o poder que nos permite usar o conhecimento para nosso próprio benefício e para o benefício dos outros.”

Thomas J. Watson

dos anos 1960 que agora estão envelhecendo fizeram um último esforço para ter seus filhos antes do fim da idade fértil. Além disso, os indivíduos da geração X (*gen-Xers*) entraram em sua idade fértil. Os dados do senso podem então ser ligados a outras partes de informações aparentemente não relacionadas. Por exemplo, o número atual de professores da escola primária que se aposentarão durante a próxima década, o número de estudantes que estarão se formando no primário e secundário, a pressão sobre os políticos para baixar os impostos e, portanto, limitar os aumentos de salários aos professores. Todas essas informações podem ser combinadas para formular uma representação do conhecimento – haverá uma pressão significativa sobre o sistema de educação nos Estados Unidos no início do século 21 e essa pressão continuará por muitas décadas. Usando esse conhecimento, pode surgir uma oportunidade de negócios. Pode haver uma oportunidade significativa de desenvolver novos modos de aprendizado mais eficazes e menos dispendiosos do que a forma atual.

O futuro do software leva a sistemas que processam conhecimento. Estivemos processando dados usando computadores por mais de 50 anos e extraindo informações por mais de três décadas. Um dos desafios mais importantes para a comunidade de engenharia de software é criar sistemas que possibilitem o próximo passo ao longo do espectro – sistemas que extraem conhecimento dos dados e informações de maneira prática e benéfica.

32.4 A VISÃO NO LONGO PRAZO

Na Seção 32.3, sugeri que o futuro leva a sistemas que “processam conhecimento”. Mas o futuro da computação em geral e dos sistemas baseados em software em particular podem levar a eventos consideravelmente mais profundos.

Em um fascinante livro, leitura obrigatória para todo profissional envolvido em tecnologias da computação, Ray Kurzweil [Kur05] afirma que já chegamos a uma época em que “o ritmo das mudanças tecnológicas será tão rápido, seu impacto tão profundo, que a vida humana será transformada irreversivelmente”. Kurzweil³ apresenta um argumento convincente de que estamos atualmente no “joelho” de uma curva de crescimento exponencial que nos levará a enormes avanços na capacidade de computação durante as próximas duas décadas. Se combinados com avanços equivalentes em nanotecnologia, genética e robótica, pode-se chegar a uma época nos meados deste século em que a distinção entre humanos (como os conhecemos hoje) e máquinas começa a se confundir – época na qual a evolução humana se acelera de modo assustador (para alguns) e espetacular (para outros).

Aproximadamente em 2030, Kurzweil afirma que a capacidade de computação e os requisitos de software serão suficientes para modelar todo aspecto do cérebro humano – todas as conexões físicas, processos analógicos e camadas químicas. Quando isso ocorrer, os seres humanos terão atingido um “poderoso AI (inteligência artificial)”, e, em consequência disso, máquinas que realmente pensam (usando os termos convencionais de hoje). Mas haverá uma diferença fundamental. Os processos do cérebro humano são excessivamente complexos e fracamente ligados às fontes externas de informação. Eles também são lentos em computação, mesmo em comparação com a tecnologia atual. Se ocorrer a emulação completa do cérebro humano, o “pensamento” ocorrerá a velocidades milhares de vezes maiores do que seu correspondente humano com conexões íntimas a um mar de informações (pense na Web de hoje como um exemplo primitivo). O resultado é... Bem, é tão fantástico que é melhor deixar que o próprio Kurzweil descreva.

É importante notar que nem todos acreditam que o futuro que Kurzweil descreve é uma coisa boa. Em um ensaio agora famoso intitulado “O futuro não precisa de nós” (*The Future Doesn't Need Us*), Bill Joy [Joy00], um dos fundadores da Sun Microsystems, diz que a “robótica, a enge-

3 É importante notar que Kurzweil não é um escritor de ficção científica comum ou um futurista sem portfólio. Ele é um tecnólogo sério que (segundo a Wikipedia) “tem sido um pioneiro nos campos do reconhecimento óptico de caractere (OCR), síntese texto-para-fala, tecnologia de reconhecimento de voz e instrumentos de teclado eletrônico”.

nharia genética e a nanotecnologia estão fazendo dos humanos uma espécie ameaçada”. Seus argumentos prevendo uma antiutopia tecnológica representam um contra-argumento ao futuro utópico previsto por Kurzweil. Ambos devem ser considerados seriamente quando os engenheiros de software assumem um dos papéis importantes definindo um futuro para a raça humana.

32.5 A RESPONSABILIDADE DO ENGENHEIRO DE SOFTWARE

A engenharia de software evoluiu tornando-se uma profissão mundialmente respeitada. Como profissionais, engenheiros de software devem se orientar por um código de ética que rege o trabalho que fazem e os produtos que criam. Uma força-tarefa da ACM/IEEE-CS (ACM/IEEE-CS Joint Task Force) produziu um código de ética e práticas profissionais da engenharia de software (*Software Engineering Code of Ethics and Professional Practices*) (Versão 5.1). O código declara[ACM98]:

Engenheiros de software deverão se comprometer a fazer da análise, especificação, projeto, desenvolvimento, teste e manutenção de software uma profissão benéfica e respeitada. De acordo com seu compromisso com a saúde, segurança e bem-estar do público, engenheiros de software deverão adotar os oito princípios a seguir:

WebRef

Uma discussão completa sobre código de ética ACM/IEEE pode ser encontrada em seeri.etsu.edu/Codes/default.shtm.

1. PÚBLICO – Engenheiros de software deverão agir consistentemente com o interesse público.
2. CLIENTE E EMPREGADOR – Engenheiros de software deverão agir de acordo com o melhor interesse de seu cliente e empregador consistente com o interesse público.
3. PRODUTO – Engenheiros de software deverão garantir que seus produtos e modificações relacionadas atinjam os mais altos padrões profissionais possíveis.
4. JULGAMENTO – Engenheiros de software deverão manter a integridade e independência de seu julgamento profissional.
5. GESTÃO – Gerentes de engenharia de software e líderes devem aderir e promover uma abordagem ética para a gestão do desenvolvimento e manutenção de software.
6. PROFISSÃO – Engenheiros de software devem prestigiar a integridade e reputação da profissão consistente com o interesse público.
7. COLEGAS – Engenheiros de software deverão ser justos e solidários com seus colegas.
8. INDIVIDUALMENTE – Engenheiros de software devem participar do contínuo aprendizado referente à prática de sua profissão e deverão promover uma abordagem ética à prática da profissão.

Embora cada um desses oito princípios seja igualmente importante, surge um tema mais importante: um engenheiro de software deve trabalhar no interesse público. Em nível pessoal, um engenheiro de software deve se guiar pelas seguintes regras:

- Nunca roubar dados para obter vantagens pessoais.
- Nunca distribuir ou vender informações proprietárias obtidas como parte de seu trabalho em um projeto de software.
- Nunca destruir ou modificar maliciosamente os programas, arquivos ou dados de outra pessoa.
- Nunca violar a privacidade de um indivíduo, um grupo ou uma organização.
- Nunca invadir um sistema por esporte ou por lucro.
- Nunca criar ou promover um vírus ou verme de computador.
- Nunca usar a tecnologia de computação para facilitar a discriminação ou assédio.

Durante a última década, certos membros da indústria de software formaram um *lobby* por uma legislação de proteção que [SEE03]: (1) permite às empresas liberar software sem revelar os defeitos conhecidos, (2) isenta os desenvolvedores da responsabilidade por quaisquer danos

resultantes desses defeitos conhecidos, (3) impede que outros descubram defeitos sem permissão do desenvolvedor original, (4) permite a incorporação de software “autoajuda” dentro do produto que pode desabilitar (via comando remoto) a operação do produto, e (5) isenta os desenvolvedores do software com “autoajuda” de danos se o software for desabilitado por um terceiro.

Como toda legislação, o debate em geral concentra-se em aspectos políticos, não tecnológicos. No entanto, muitas pessoas (até mesmo eu) pensam que a legislação protetora, se formulada incorretamente, entra em conflito com o código de ética da engenharia de software, isentando indiretamente os engenheiros de sua responsabilidade de produzir software de alta qualidade.

32.6 COMENTÁRIO FINAL

Já se passaram 30 anos desde que comecei a trabalhar na primeira edição deste livro. Ainda me lembro sentado à minha mesa como jovem professor, escrevendo o manuscrito de um livro sobre um assunto com o qual poucas pessoas se importavam e ainda menos pessoas realmente entendiam. Lembro-me das cartas de rejeição dos editores, que argumentavam (educadamente, mas com firmeza) que nunca haveria um mercado para um livro sobre “engenharia de software”. Felizmente, a McGraw-Hill decidiu tentar,⁴ e o resto, como se costuma dizer, é história.

Durante os últimos 30 anos, este livro mudou significativamente – em escopo, em tamanho, em estilo e em conteúdo. Assim como a engenharia de software, cresceu e (espero) amadureceu durante os últimos anos.

Uma abordagem de engenharia para o desenvolvimento de software de computador é hoje sabedoria convencional. Embora o debate continue sobre o “paradigma certo”, a importância da agilidade, o grau de automação e os métodos mais eficazes, os princípios subjacentes de engenharia de software são agora aceitos em toda a indústria. Por que, então, só recentemente temos visto uma adoção mais ampla?

A resposta, penso, está na dificuldade da transição tecnológica e na mudança cultural que a acompanha. Apesar de muitos de nós apreciarem a necessidade de uma disciplina de engenharia para software, lutamos contra a inércia da prática anterior e nos defrontamos com novos domínios de aplicação (e os desenvolvedores que trabalham nelas) que parecem prontos para repetir os mesmos erros do passado. Para facilitarmos a transição, precisamos de muitas coisas – um processo de software ágil, adaptável e sensível; métodos mais eficazes; ferramentas mais poderosas; melhor aceitação pelos profissionais e apoio dos gerentes; e uma boa dose de educação.

Você pode não concordar com todas as abordagens descritas neste livro. Algumas das técnicas e opiniões são controvertidas; outras devem ser ajustadas para funcionar bem em diferentes ambientes de desenvolvimento de software. No entanto, espero sinceramente que *Engenharia de software: uma abordagem profissional* tenha delineado os problemas que enfrentamos, demonstrado o poder dos conceitos de engenharia de software e tenha proporcionado uma estrutura de métodos e ferramentas.

À medida que avançamos pelo século 21, o software continua a ser o produto e a indústria mais importantes no cenário mundial. Seu impacto e importância têm um amplo alcance. Contudo, uma nova geração de desenvolvedores de software deve enfrentar muitos dos mesmos desafios que gerações anteriores enfrentaram.

Esperemos que as pessoas que enfrentam esse desafio – os engenheiros de software – tenham a sabedoria para desenvolver sistemas que melhorem a condição humana.

⁴ Na realidade, o mérito vai para Peter Freeman e Eric Munson, que convenceram a McGraw-Hill de que valia a pena tentar. Depois de um milhão de cópias, é justo dizer que tomaram uma boa decisão.